

Source Code

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

products = [
    "Milk", "Cereal", "Bread", "Eggs", "Butter", "Yogurt",
    "Diapers", "Baby Wipes", "tissue", "fruit Juice", "Chips", "Soda"
]
num_products = len(products)
grid_shape = (3, 4)

affinity_matrix = np.zeros((num_products, num_products))
def add_affinity(i, j, score):
    affinity_matrix[i, j] = score
    affinity_matrix[j, i] = score

add_affinity(0, 1, 8)
add_affinity(0, 2, 6)
add_affinity(0, 3, 7)
add_affinity(2, 4, 6)
add_affinity(3, 4, 5)
add_affinity(0, 5, 5)
add_affinity(6, 7, 10)
add_affinity(6, 8, 9)
add_affinity(8, 10, 8)
add_affinity(8, 11, 7)
add_affinity(9, 10, 6)
add_affinity(10, 11, 8)

height_multipliers = np.array([0.7, 1.6, 1.4, 0.5])

def evaluate_2d_layout(position):
    flat_order = np.argsort(position)
    layout_grid = np.zeros(grid_shape, dtype=int)
    idx = 0
    for r in range(grid_shape[0]):
        for c in range(grid_shape[1]):
            layout_grid[r, c] = flat_order[idx]
            idx += 1

    score = 0
    for r in range(grid_shape[0]):
        for c in range(grid_shape[1]):
            current_product = layout_grid[r, c]

            if current_product in [6, 8, 11]:
                score += 12 if c == 3 else 0
            else:
                score += height_multipliers[c] * 6

            if c < grid_shape[1] - 1:
                right_neighbor = layout_grid[r, c+1]
                score += affinity_matrix[current_product, right_neighbor]

            if r < grid_shape[0] - 1:
                down_neighbor = layout_grid[r+1, c]
                score += affinity_matrix[current_product, down_neighbor] * 0.5

    return score

num_particles = 40
max_iterations = 80
w, c1, c2 = 0.5, 1.5, 1.5

X = np.random.uniform(0, 10, (num_particles, num_products))
V = np.random.uniform(-1, 1, (num_particles, num_products))

pbest_X = np.copy(X)
pbest_fitness = np.array([evaluate_2d_layout(p) for p in X])

gbest_idx = np.argmax(pbest_fitness)
```

```

gbest_X = np.copy(pbest_X[gbest_idx])
gbest_fitness = pbest_fitness[gbest_idx]

for iteration in range(max_iterations):
    for i in range(num_particles):
        r1, r2 = np.random.rand(num_products), np.random.rand(num_products)
        V[i] = w * V[i] + c1 * r1 * (pbest_X[i] - X[i]) + c2 * r2 * (gbest_X - X[i])
        X[i] += V[i]

        current_fitness = evaluate_2d_layout(X[i])
        if current_fitness > pbest_fitness[i]:
            pbest_fitness[i] = current_fitness
            pbest_X[i] = np.copy(X[i])

    best_particle = np.argmax(pbest_fitness)
    if pbest_fitness[best_particle] > gbest_fitness:
        gbest_fitness = pbest_fitness[best_particle]
        gbest_X = np.copy(pbest_X[best_particle])

final_flat_order = np.argsort(gbest_X)
final_grid_names = np.zeros(grid_shape, dtype=object)
final_grid_indices = np.zeros(grid_shape, dtype=int)

idx = 0
for r in range(grid_shape[0]):
    for c in range(grid_shape[1]):
        final_grid_names[r, c] = products[final_flat_order[idx]]
        final_grid_indices[r, c] = final_flat_order[idx]
        idx += 1

plt.figure(figsize=(12, 6))
sns.heatmap(final_grid_indices, annot=False, cmap="YlGnBu", cbar=False,
            linewidths=5, linecolor="white")

for r in range(grid_shape[0]):
    for c in range(grid_shape[1]):
        plt.text(c + 0.5, r + 0.5, final_grid_names[r, c],
                horizontalalignment='center', verticalalignment='center',
                fontsize=12, fontweight='bold', color='black')

plt.xticks([0.5, 1.5, 2.5, 3.5], ['Top Shelf', 'Eye Level Upper', 'Eye Level Lower', 'Bottom Shelf'], fontsize=11)
plt.yticks([0.5, 1.5, 2.5], ['Aisle 1', 'Aisle 2', 'Aisle 3'], rotation=0, fontsize=11)
plt.title(f"PSO Multi-Department Supermarket Layout Blueprint\n(Optimized Basket Score: {gbest_fitness:.1f})",
        fontsize=14, fontweight='bold', pad=20)

plt.tight_layout()
plt.show()

```

Output

**

PSO Multi-Department Supermarket Layout Blueprint
(Optimized Basket Score: 134.2)

